

ورشة GIT وGITHUB

حل التعارضات ونشر الموقع

Session 05 · Merge Conflicts & GitHub Pages

هدف اليوم — ثلاث مهارات يطلبها سوق العمل

- حل التعارضات بثقة: نفهم لماذا حدث، ولا نخاف منه
- إدارة الإصدارات: وسم الإصدار الأول من المشروع
- النشر: الموقع يظهر على رابط عام يراه أي شخص
- التعارض ليس فشلاً: هو دعوة من Git لاتخاذ قرار واعٍ

اليوم **الدمج** لا يُنهي إلا بعد فهمه: كل تعارض حلّيته ستشرحه في تقرير التسليم بجملة واحدة واضحة.

Conflict markers — بنية علامات التعارض في الملف

```

data/members-index.json

"files": [
  <<<<<<< HEAD
    "leen-docs.json",
  =====
    "omar-qa.json",
  >>>>>>> feature/omar-qa-card
    "sara-design.json"
]

```

A) نسختك الحالية (الفرع الذي تستهدفه)
 your current branch — "ours"

C) الفاصل: نهاية نسختك وبداية الأخرى

B) النسخة القادمة (الفرع المدموج)
 the incoming branch — "theirs"

ثم `git merge --continue` ثم `git add` الحل: احذف الأسطر الثلاثة للعلامات، واترك النتيجة الصحيحة، ثم

- شرطان معاً: تعديلان متوازيان + نفس الأسطر
- تعديلان في ملفين مختلفين ← لا تعارض
- تعديلان في نفس الملف لكن أسطراً مختلفة ← لا تعارض (يحدث دمج تلقائي)
- تعديلان في نفس السطر ← تعارض

```
$ git switch main && git pull --ff-only
$ git switch -c sandbox/conflict-<team>

$ nano data/site-config.json      # دليل صف 2026 : الفريق الأول
$ node tests/validate-members.mjs
$ git add data/site-config.json
$ git commit -m "content(config): set team headline (team A)"
$ git push -u origin sandbox/conflict-<team>
```

- تكرار التعارض بشكل مقصود في بيئة آمنة هو أسرع طريق لإتقانه
- ملف `data/site-config.json` هو ميدان التدريب: يتغير بسرعة، ولن يضر الموقع

```
$ git switch main && git pull --ff-only
$ git merge sandbox/conflict-<team>
Auto-merging data/site-config.json
CONFLICT (content): Merge conflict in data/site-config.json
Automatic merge failed; fix conflicts and then commit the result.

$ git status -sb
## main
UU data/site-config.json
```

```
{
  "siteName": "دليل فريق الصف",
<<<<<< HEAD
  "h1Title": "دليل صف 2026",
=====
  "h1Title": "دليل فريق الصف",
>>>>>> sandbox/conflict-team-a
  "contactEmail": "team@example.com"
}
```

العلامة	تعني
HEAD >>>>>>	بداية منطقة التعديل الموجود على فرعك الحالي
=====	الخط الفاصل بين الرأيين
name <<<<<<	بداية تعديل الطرف الآخر

✘ خطأ قاتل: ترك أي من العلامات الثلاث في الملف. المشروع لن يعمل، وهذه أسرع طريقة لرفض طلبك في المراجعة.

```
{  
  "siteName": "دليل فريق الصف",  
  "h1Title": "دليل صف 2026",  
  "contactEmail": "team@example.com"  
}
```

```
$ grep -rInE "^(<{7}|={7}>{7})( |$)" --exclude-dir=.git .
```

✓ لا مخرجات = لا علامات متبقية ←

```
$ node tests/validate-members.mjs
```

```
--بطاقة, 0 خطأ, 0 تحذير 6 Result / النتيجة ---
```

```
$ git add data/site-config.json
```

```
$ git commit -m "merge: resolve site-config headline conflict"
```

```
$ git push
```

- القرار هنا: نريد عنوان «دليل صف 2026» — لأن الأحدث هو الأصح
- الحل الأفضل ليس «خذ رأيي»، بل **ماذا سيري الزائر؟**

الأداة	القيمة عند التعارض
VS Code	تلوين الطرفين وأزرار Accept Current/Incoming — مع معاينة مباشرة
GitHub Desktop	فتح الملف في محرر والخيارات الأربعة، ثم Continue merge
git diff	رؤية الفرق بنص واضح قبل أي قرار
git mergetool	فتح أداة الفروق الثلاثية (ثلاثة أعمدة) عند الحاجة

مهما استخدمت من أدوات: **القرار قرارك**، والأداة تعرض لا تحكم. اقرأ الطرفين قبل الموافقة.

```
$ git tag -a v1.0.0 -m "بطاقات الفريق العشرة + صفحة الأسئلة الشائعة"الإصدار الأول:
```

```
$ git push origin v1.0.0
```

```
To https://github.com/<org>/class-team-hub.git
```

```
* [new tag]          v1.0.0 → v1.0.0
```

```
$ git show v1.0.0 --stat --no-patch
```

```
tag v1.0.0
```

```
Tagger: Omar Al-Qasem <omar@example.com>
```

```
Date:   Tue Nov 3 11:20:14 2026 +0300
```

```
الإصدار الأول: بطاقات الفريق العشرة + صفحة الأسئلة الشائعة
```

- **الوسم معنون** (-a) يحمل: من وسم، ومتى، ولماذا — لأنه مرجع رسمي للفريق

Settings → Pages

Source: Deploy from a branch

Branch: main / (root)

→ Save

بعد دقيقة :

<https://<org>.github.io/class-team-hub/>

الصفحة قد تحتاج 1-3 دقائق حتى تظهر أول مرة. حدّث الصفحة، وإن استمر الخطأ فتتحقق من: اسم الفرع، ومجلد الإخراج، وأن `index.html` في الجذر.

عند نجاح النشر: ضع الرابط في وصف المستودع (About → Website) وفي رسالة الالتزام النهائي.

- افتح رابط الموقع من هاتف زميلك: هل تظهر البطاقات كلها؟
- نادِ ثلاثة أعضاء باسم كل واحد: هل بطاقته تظهر ببياناته الصحيحة وترتيبه الصحيح؟
- افتح `git log --oneline --graph --all` واسأل: أي عقدة جمعت عمل كل الفريق؟
- تحقق من `git tag -l` ثم `git ls-remote --tags origin`: هل الوسم على الخادم فعلاً؟

الوسم المحلي وحده **لا يُرفع تلقائياً** مع `git push` — يحتاج `git push origin v1.0.0` أو `git push origin --tags`.

- عندك الآن: مهارة يطلبها كل فريق برمجي، ومشروع حقيقي على الإنترنت باسمك
- أتقنت: الالتزام، الفرع، طلب السحب، المراجعة، المزامنة، الدمج، حل التعارض، النشر
- الخطوة التالية (اختيارية متقدمة): **الوحدة 06 — الأتمتة الاحترافية** (ws6-automation)
- فيها: GitHub Actions، حماية الفرع، CODEOWNERS، الإصدارات، وسجل التغييرات

هذه المهارات تُختبر في المقابلات العملية. احتفظ بمستودع المشروع في سيرتك، واربطه بحساب GitHub.

لا تنس: كل ما تعلمته اليوم يستخدمه المهندسون كل يوم في العمل الفعلي. الفرق بينك وبينهم اليوم: عدد المحاولات، لا الموهبة.